

Docker - Cheat Sheet

Docker is a tool for packaging applications and their dependencies into portable containers that run identically across environments. Born out of Solomon Hykes's work at dotCloud in 2013, it took over backend development workflows within a few years.

If you've ever heard "but it works on my machine", Docker is the standard fix.

What Docker Is

Three core pieces:

1. **Docker Engine:** the daemon (`dockerd`) that runs and manages containers, plus a CLI (`docker`) you use to interact with it.
2. **Images:** read-only templates that define what's inside a container.
3. **Containers:** running instances of images. Lightweight, isolated processes.

A container is a process running on the host kernel with isolated filesystem, network, and process namespaces. It feels like a tiny VM but skips the hypervisor.

Containers vs VMs

Both isolate workloads. They do it at different levels.

Aspect	Container	VM
Isolation level	OS process	Hardware (hypervisor)
Boot time	Seconds	Minutes
Disk size	MB to a few hundred MB	GB
Kernel	Shared with host	Its own
Density	100s per host	10s per host
Use case	Apps, microservices	Different OS, strong isolation

Containers share the host kernel, so a Linux container needs a Linux kernel. Docker Desktop on Mac and Windows runs a lightweight Linux VM under the hood to make this work.

Core Concepts

Image

A read-only filesystem snapshot plus metadata about how to run it. Built from a `Dockerfile`. Stored locally or pulled from a registry like Docker Hub.

```
docker pull nginx:1.27
```

Images have a name and a tag (`nginx:1.27`). No tag means `:latest`, which is dangerous in production.

Container

A running (or stopped) instance of an image. Each `docker run` creates a new container.

```
docker run nginx:1.27
```

Containers are ephemeral by default. When they stop, their writable layer is gone unless you mounted a volume.

Volume

Persistent storage that survives container restarts and deletions. Lives outside the container's filesystem.

```
docker volume create db-data  
docker run -v db-data:/var/lib/postgresql postgres:16
```

Network

How containers talk to each other and the outside world. Docker creates a default bridge network. You can create more for isolation.

Registry

Where images live. Docker Hub is the default public one. Private registries: AWS ECR, GCP Artifact Registry, GitHub Container Registry, Harbor.

Common Commands

Command	What it does
<code>docker run <image></code>	Create and start a new container
<code>docker run -d <image></code>	Detached (background) mode
<code>docker run -it <image> bash</code>	Interactive with a shell
<code>docker ps</code>	List running containers
<code>docker ps -a</code>	List all containers (including stopped)
<code>docker stop <id></code>	Stop a running container
<code>docker rm <id></code>	Remove a stopped container
<code>docker rm -f <id></code>	Force remove (stop and remove)
<code>docker images</code>	List local images
<code>docker rmi <image></code>	Remove an image
<code>docker logs <id></code>	View container logs
<code>docker logs -f <id></code>	Follow logs (like <code>tail -f</code>)
<code>docker exec -it <id> bash</code>	Run a command in a running container
<code>docker build -t myapp .</code>	Build an image from a Dockerfile
<code>docker pull <image></code>	Download an image
<code>docker push <image></code>	Upload an image to a registry
<code>docker inspect <id></code>	Show detailed config and state
<code>docker stats</code>	Live CPU and memory usage
<code>docker system prune</code>	Clean up unused containers, images, networks

Port mapping

```
docker run -p 8080:80 nginx
```

Maps host port 8080 to container port 80. Without `-p`, the container's ports are unreachable from outside.

Environment variables

```
docker run -e DATABASE_URL=postgres://... myapp
```

Or load from a file:

```
docker run --env-file .env myapp
```

Naming

```
docker run --name web nginx
```

Without `--name`, Docker generates a random one like `elated_einstein` or `nostalgic_brahmagupta`.

Dockerfile

A text file that describes how to build an image. Each instruction creates a new layer.

```
# Base image
FROM eclipse-temurin:21-jdk-alpine

# Set working directory
WORKDIR /app

# Copy dependency manifests first (for cache)
COPY pom.xml .

# Download dependencies (cached if pom.xml unchanged)
RUN ./mvnw dependency:go-offline

# Copy source code
COPY src ./src

# Build the application
RUN ./mvnw package -DskipTests

# Expose the port
EXPOSE 8080

# What to run when the container starts
ENTRYPOINT ["java", "-jar", "target/myapp.jar"]
```

Common instructions:

Instruction	Purpose
FROM	Base image to start from
WORKDIR	Set the working directory inside the image
COPY	Copy files from host into the image
ADD	Like COPY, plus URL and tarball auto-extraction (prefer COPY)
RUN	Run a command at build time, layer by layer
ENV	Set an environment variable for the container
ARG	Build-time variable (not in the running container)
EXPOSE	Document which ports the container listens on (doesn't publish)
VOLUME	Declare a mount point
USER	Switch to a non-root user
ENTRYPOINT	The main process the container runs
CMD	Default arguments to ENTRYPOINT, or default command
HEALTHCHECK	How Docker should check the container's health

ENTRYPOINT vs CMD :

- ENTRYPOINT ["java", "-jar", "app.jar"] is the fixed command.
- CMD ["--profile=prod"] is default args, overridable at docker run time.
- Together: docker run myapp --profile=dev runs java -jar app.jar --profile=dev .

Image Layers and Caching

Each instruction in a Dockerfile creates a layer. Layers are cached. If a layer's inputs haven't changed, Docker reuses the cache.

Implication: order your Dockerfile from least to most frequently changing.

Bad (cache busts on every code change):

```
FROM node:20
WORKDIR /app
COPY . .
RUN npm install
RUN npm run build
```

Good (dependency layer caches as long as `package.json` is stable):

```
FROM node:20
WORKDIR /app
COPY package.json package-lock.json ./
RUN npm install
COPY . .
RUN npm run build
```

The `npm install` step is the slow one. Putting `COPY package.json` before `COPY .` keeps the install cached when only source files change.

Image size

Each `RUN` adds a layer. Chain commands with `&&` to keep them in one layer:

```
RUN apt-get update \
    && apt-get install -y --no-install-recommends curl \
    && rm -rf /var/lib/apt/lists/*
```

Use a `.dockerignore` file to skip files you don't want in the image:

```
.git
node_modules
target
*.log
.env
```

Multi-Stage Builds

Use one image to build, copy the artifacts into a smaller image to run. Cuts image size dramatically.

```
# Stage 1: build
FROM eclipse-temurin:21-jdk-alpine AS build
WORKDIR /app
COPY pom.xml .
COPY src ./src
RUN ./mvnw package -DskipTests

# Stage 2: runtime
FROM eclipse-temurin:21-jre-alpine
WORKDIR /app
COPY --from=build /app/target/myapp.jar app.jar
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app.jar"]
```

Final image includes only the JRE and the JAR. The Maven build tools, JDK, and source code stay in the build stage. Image size drops from around 600 MB to around 200 MB.

Volumes

Three ways to mount data into a container.

Named volume (preferred for app data)

```
docker volume create db-data
docker run -v db-data:/var/lib/postgresql/data postgres
```

Docker manages the storage. Easy to back up. Easy to share across containers.

Bind mount (for source code in dev)

```
docker run -v $(pwd)/src:/app/src myapp
```

Maps a host directory directly into the container. Good for dev because file changes show up immediately. Bad for portability because the host path is specific.

tmpfs (in-memory)

```
docker run --tmpfs /tmp myapp
```

Lives in RAM, gone when the container stops. Good for caches or sensitive data.

Networking

Docker creates a default bridge network. Containers on the same user-defined bridge can reach each other by name.

```
# Create a network
docker network create app-net

# Run containers on it
docker run -d --network app-net --name db postgres
docker run -d --network app-net --name web myapp

# From the web container, `db` resolves to the postgres container
```

Common network modes:

Mode	Behavior
bridge (default)	Container gets its own IP on the bridge network
host	Container shares the host's network stack (no isolation)
none	No networking
Custom user-defined bridge	Like default but with DNS resolution between containers

Publish ports with `-p hostPort:containerPort` to reach a container from the host.

Docker Compose

Defines multi-container applications in YAML. One command to start everything.

```
# docker-compose.yml
services:
  web:
    build: .
    ports:
      - "8080:8080"
    environment:
      DATABASE_URL: postgres://db:5432/myapp
    depends_on:
      - db

  db:
    image: postgres:16
    environment:
      POSTGRES_DB: myapp
      POSTGRES_USER: user
      POSTGRES_PASSWORD: pass
    volumes:
      - db-data:/var/lib/postgresql/data

volumes:
  db-data:
```

Common commands:

Command	Purpose
<code>docker compose up</code>	Start all services
<code>docker compose up -d</code>	Detached mode
<code>docker compose down</code>	Stop and remove containers
<code>docker compose down -v</code>	Also remove volumes
<code>docker compose logs -f web</code>	Follow logs of one service
<code>docker compose exec web bash</code>	Shell into a running service
<code>docker compose build</code>	Rebuild images
<code>docker compose ps</code>	List services

Services on the same compose file share a default network where they reach each other by service name (`db` , `web`).

Best Practices

- **Pin image versions.** Use `nginx:1.27` rather than `nginx:latest`. Repeatable builds matter.
 - **Use small base images.** `alpine` variants are tiny but use musl libc. `slim` Debian variants are a bit larger but more compatible.
 - **Run as a non-root user.** Add `USER appuser` in the Dockerfile after creating the user.
 - **Use multi-stage builds.** Keep runtime images lean.
 - **One process per container.** Don't pack `nginx` + your app + a cron job in one container. Run them as separate services.
 - **Order Dockerfile by change frequency.** Stable layers first, churn last.
 - **Use `.dockerignore`.** Stops `node_modules` and secrets from sneaking in.
 - **Set resource limits.** `--memory` and `--cpus` prevent one container from eating the host.
 - **Use health checks.** `HEALTHCHECK` lets orchestrators know when to restart a sick container.
 - **Log to stdout.** Containers should write logs to stdout/stderr. Docker captures them.
-

Common Gotchas

- **latest tag is mutable.** It points to whatever was last pushed, which can break your build silently.
 - **Containers don't keep state.** Without volumes, every restart loses data.
 - **Layer cache breaks on `COPY` . early.** Put dependency installation before copying all source.
 - **Don't store secrets in images.** They get baked into layers, visible to anyone with the image. Use environment variables or a secret store.
 - **docker build runs from the build context.** Files outside the context (above the directory you pass) can't be copied in.
 - **Bind mounts can override container files.** If you bind-mount a directory that the image already populated, the image's content gets hidden.
 - **Containers run as root by default.** Container root has fewer capabilities than host root, but still risky. Use `USER`.
 - **EXPOSE doesn't publish ports.** It's documentation. You still need `-p` at run time.
 - **Docker Desktop has resource limits.** On Mac and Windows, the underlying VM gets a fixed slice of CPU and RAM. Check settings if containers feel slow.
 - **docker system prune removes images.** Use with care. You might lose images you actually want.
-

Quick Reference

Concept	Key fact
Image	Read-only template, built from a Dockerfile
Container	Running instance of an image
Volume	Persistent storage outside the container filesystem
Network	How containers talk to each other and the outside
Registry	Where images are stored and shared
<code>docker run</code>	Create and start a container
<code>docker build</code>	Build an image from a Dockerfile
<code>docker exec</code>	Run a command in a running container
<code>docker logs</code>	View container output
<code>docker compose</code>	Manage multi-container apps from one YAML file
Multi-stage build	Build in one image, run in a smaller one
<code>.dockerignore</code>	Files to skip when building